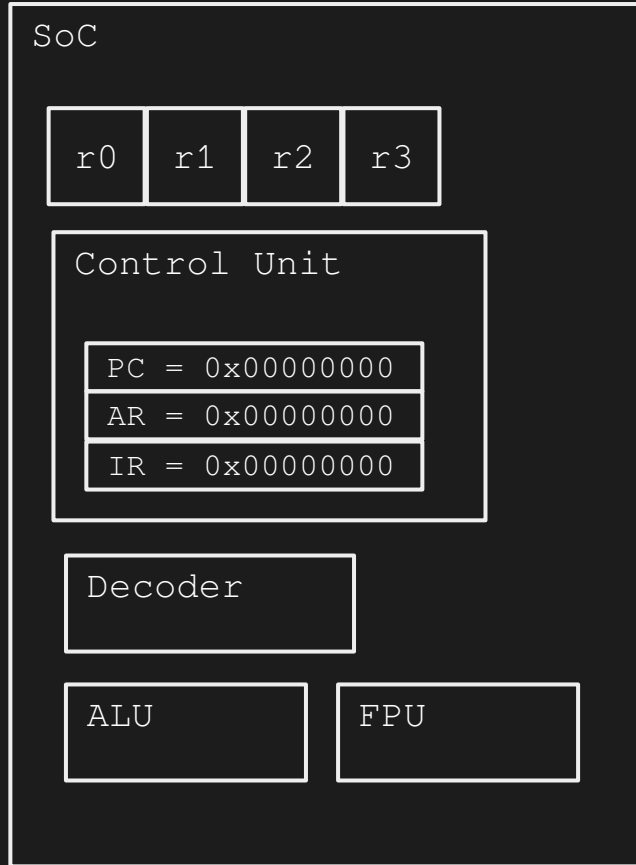


Machine Code

Binaries



Execution

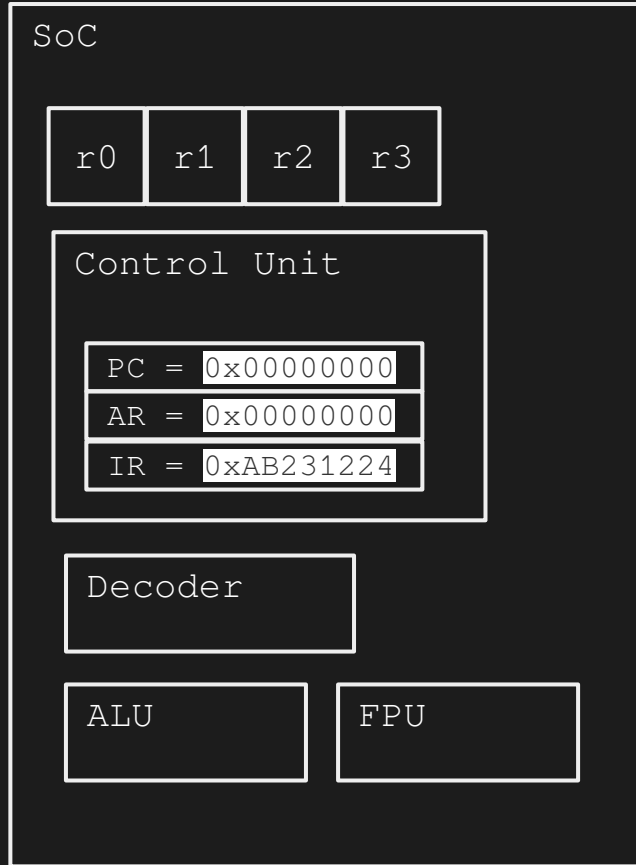


data addr ctrl

RAM

0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

Fetch

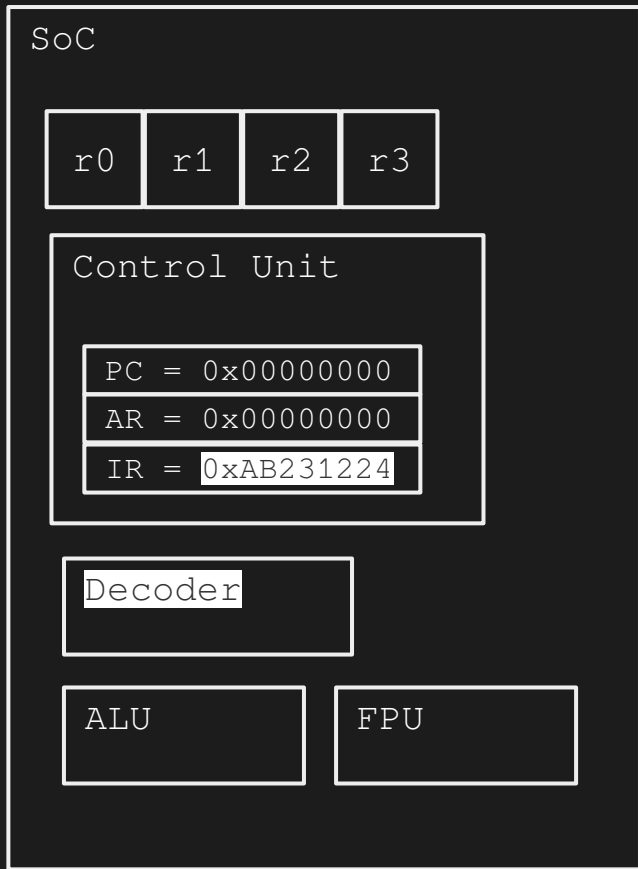


.	.	.
data	addr	ctrl

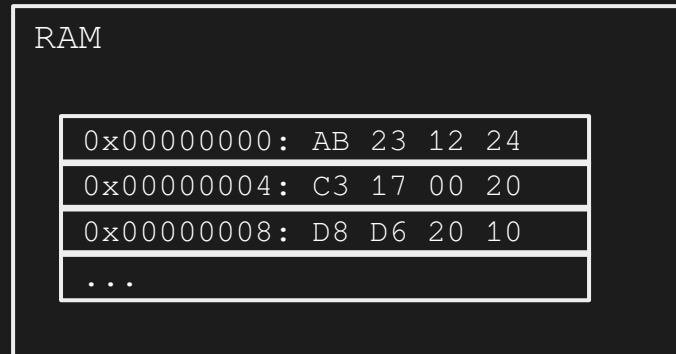
RAM

0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

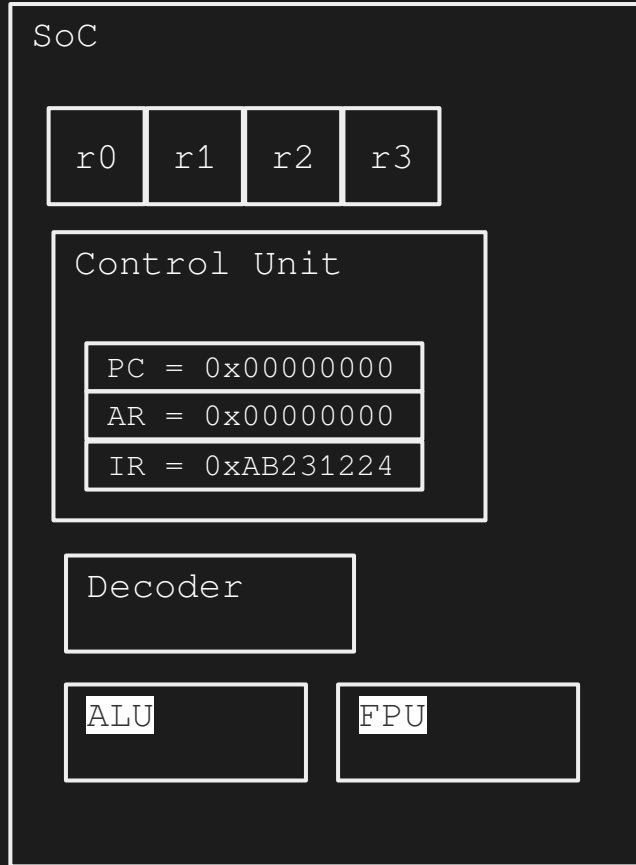
Decode



```
data addr ctrl
```



Execute



.	.	.
data	addr	ctrl

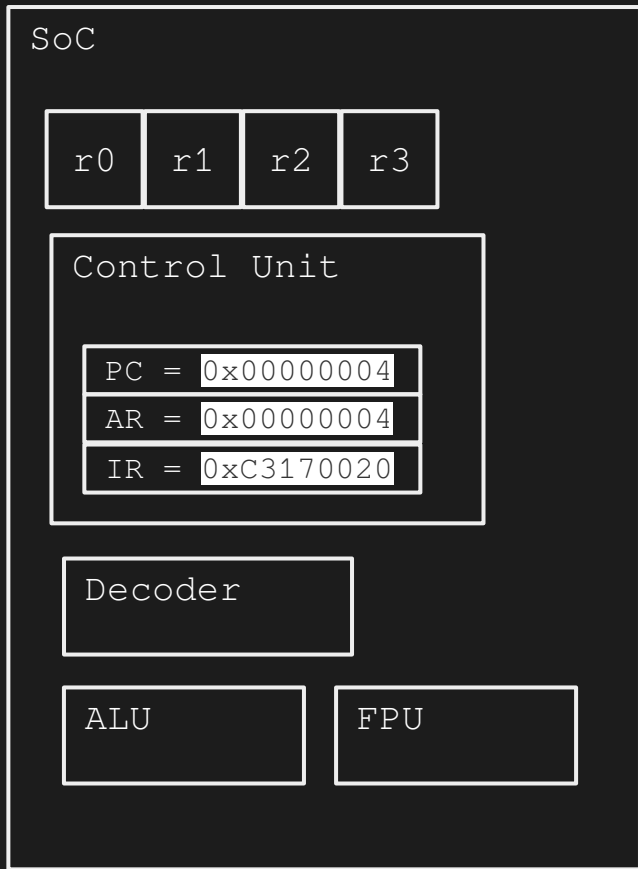
Three vertical dashed lines represent the data, address, and control buses. Each line starts with a dot at the top and ends with a dot at the bottom, with the labels 'data', 'addr', and 'ctrl' respectively.

RAM

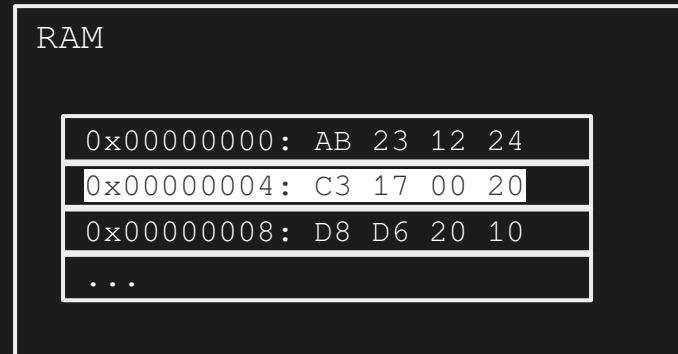
0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

The RAM block is a rectangle containing a table of memory addresses and their corresponding data values. The addresses are 0x00000000, 0x00000004, and 0x00000008, followed by an ellipsis. The data values are AB 23 12 24, C3 17 00 20, and D8 D6 20 10.

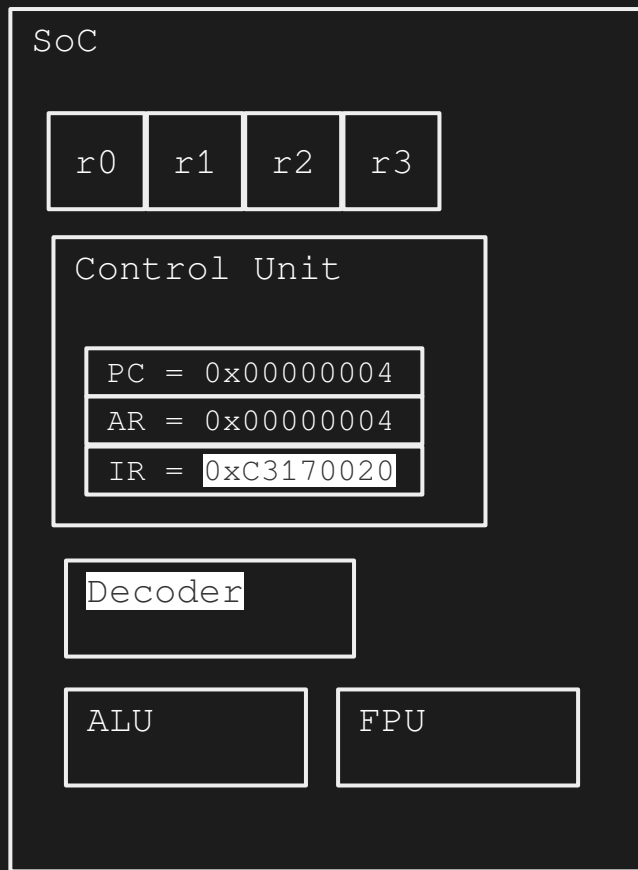
Fetch



```
data addr ctrl
```



Decode

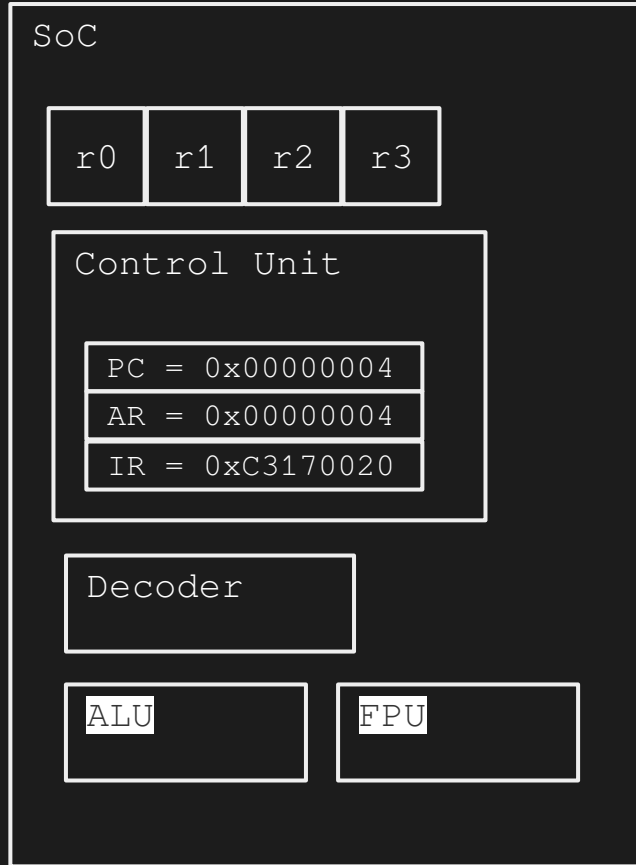


.	.	.
data	addr	ctrl

RAM

0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

Execute



data addr ctrl

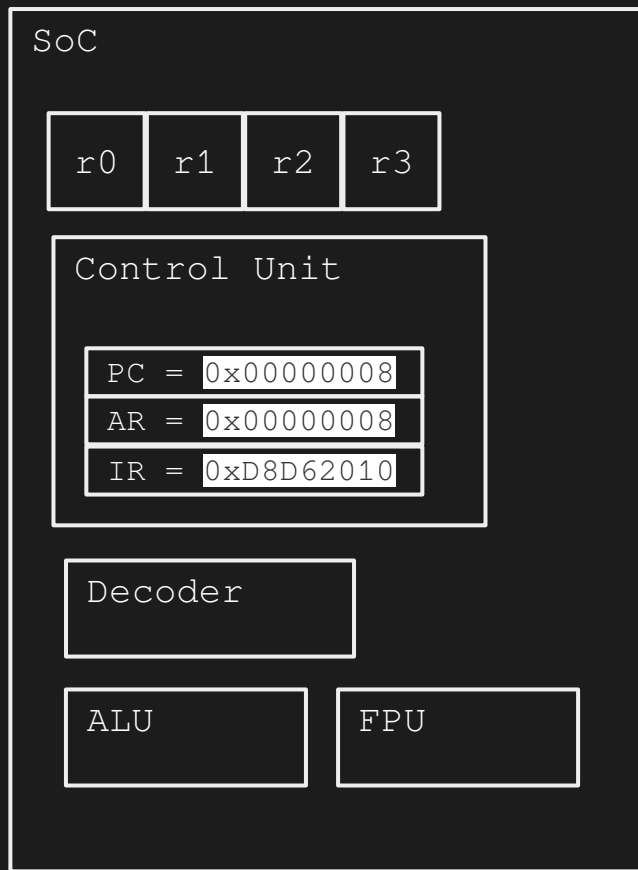
Three vertical dashed lines represent the system buses: data, address, and control. Each line starts with a small dot at the bottom, corresponding to the labels 'data', 'addr', and 'ctrl' respectively.

RAM

0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

The diagram shows a portion of RAM with four rows. Each row contains a hexadecimal address followed by four hexadecimal values. The addresses are 0x00000000, 0x00000004, 0x00000008, and an ellipsis indicating further memory.

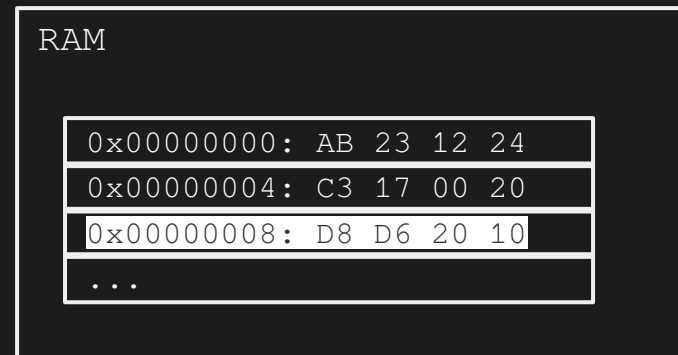
Fetch



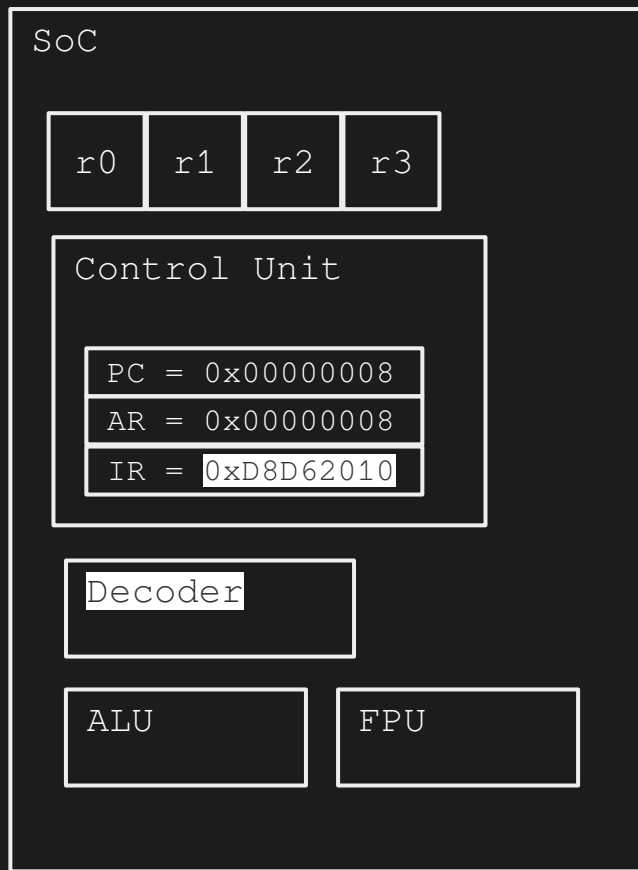
```

data  addr  ctrl

```



Decode

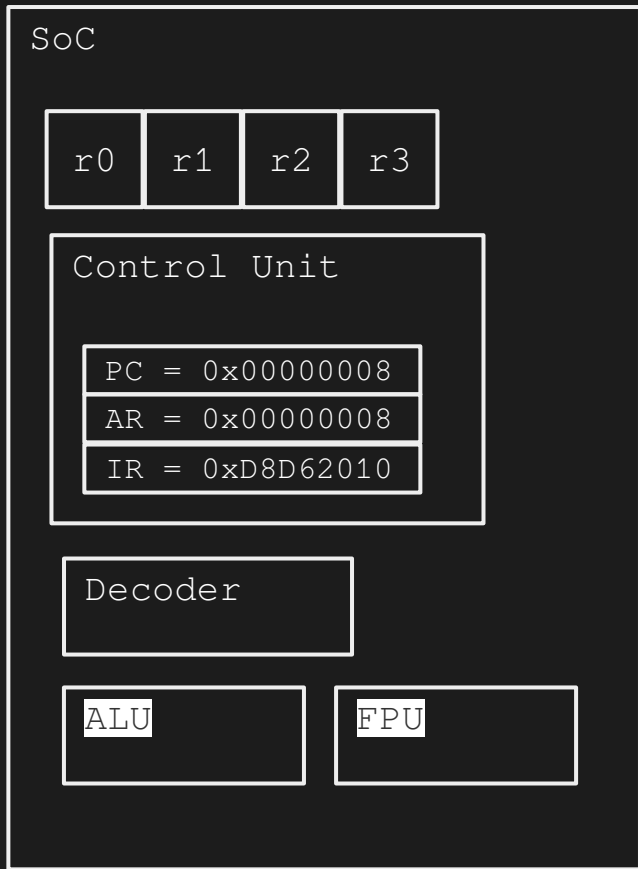


.	.	.
data	addr	ctrl

RAM

0x00000000:	AB	23	12	24
0x00000004:	C3	17	00	20
0x00000008:	D8	D6	20	10
...				

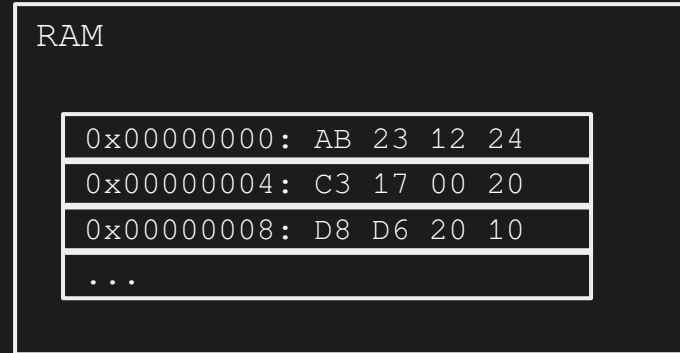
Execute



```

data  addr  ctrl

```



Operation Codes

83 C0 01 → **add** eax, 1

→ 83 = arithme

x86-64 Encoding

B8 01 00 00 00 → **mov** eax, 1

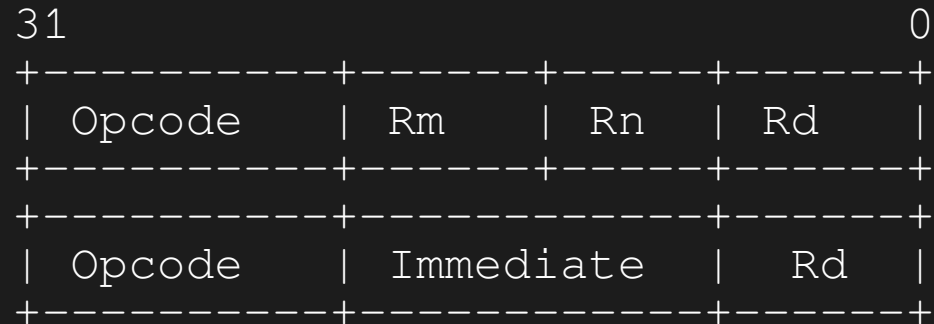
1 .. 15 bytes

Prefix	Opcode	ModRM	SIB	Imm
--------	--------	-------	-----	-----

- **Prefix** = modifies instruction (REX prefix)
- **Opcode** = actual operation
- **ModRM** = Register, Memory, ...
- **Scale-Index-Base** = Complex arithmetic, e.g.: $[rax+rbx*4]$
- **Immediate** = Constant value

aarch64 Encoding

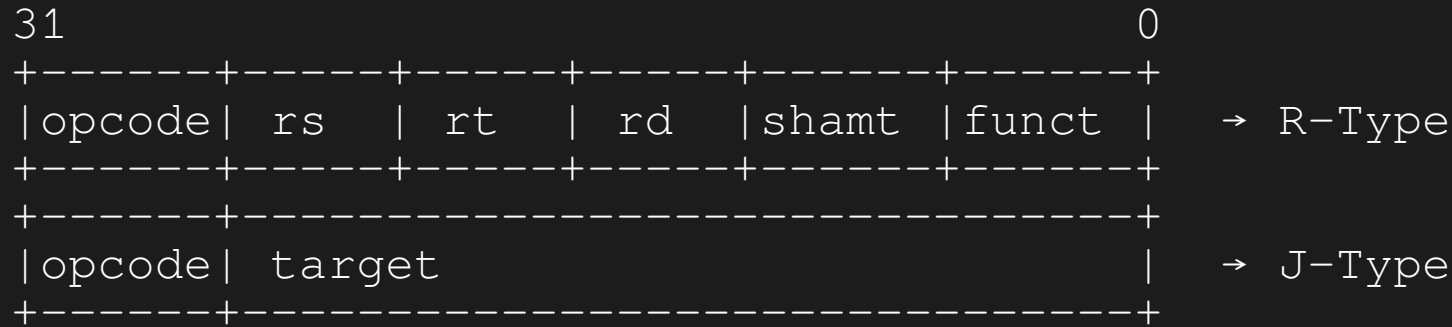
20 00 02 8B → **add** x0, x1, x2



- **Opcode** = the actual operation
- **R*** = registers
- **Immediate** = constant value

mips64 Encoding

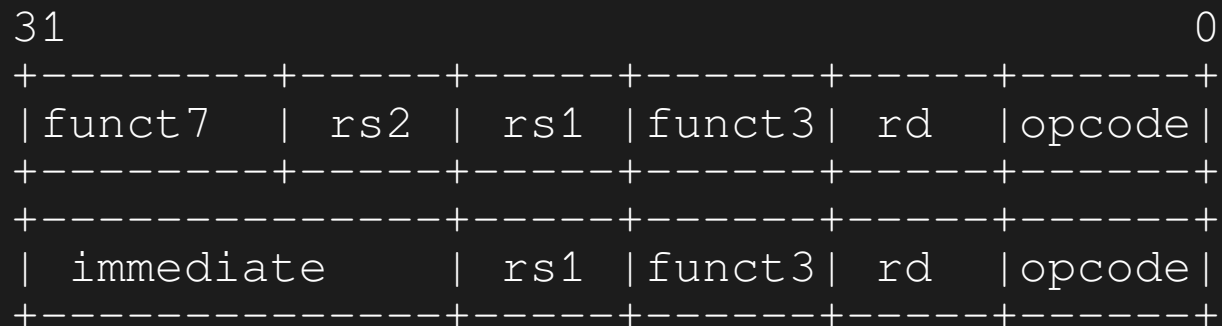
01 2A 40 20 → **add** \$t0, \$t1, \$t2



- **opcode** = the actual operation
- **r*** = registers
- **shamt** = shift amount
- **funct** = function

risc64 Encoding

00 55 05 13 → **addi** a0, a0, 5



- **opcode** = the actual operation
- **immediate** = constant value
- **r*** = registers
- **funct*** = function

The Disassembler

```
B8 01 00 00 00 → mov eax, 0x01
EB 05          → jmp 0x0000000000000007
90            → nop
90            → nop
90            → nop
90            → nop
90            → nop
C3            → ret
```

Linear Disassembler

```
offset = 0;
while (offset < file_size)
{
    decode_instruction(offset);
    offset += instruction_length;
}
```

```
// B8 01 00 00 00 → mov eax, 1
// 90              → nop
// 90              → nop
// C3              → ret
```

Code + Data Mixup

```
→ EB 06          ; jmp skip
→ 53 45 43 52 45 54 ; "SECRET"
→ C3             ; ret
```

```
start:
    jmp skip
```

```
secret:
    db 'S'
    db 'E'
    db 'C'
    db 'R'
    db 'E'
    db 'T'
```

```
skip:
    ret
```

0000:	EB 06	jmp 0x08
0002:	53	push rbx
0003:	45	????
0004:	43	????
0005:	52	push rdx
0006:	45	????
0007:	54	push rsp
0008:	C3	ret

Recursive Disassembler

```
→ EB 06          ; jmp skip
→ 53 45 43 52 45 54 ; "SECRET"
→ C3             ; ret
```

start:	0000: EB 06	jmp 0x08
jmp skip	0002: 53	...
	0003: 45	...
secret:	0004: 43	...
db 'S'	0005: 52	...
db 'E'	0006: 45	...
db 'C'	0007: 54	...
db 'R'	0008: C3	ret
db 'E'		
db 'T'		
skip:		
ret		

Questions?